XPLOG++: Dynamic Observability for Distributed Edges

Project-II (CS57004) report submitted to

Indian Institute of Technology Kharagpur

in partial fulfilment for the award of the degree of

Master of Technology

in

Computer Science and Engineering

by

Aman Sharma

(20CS30063)

Under the supervision of

Professor Sandip Chakraborty

Department of Computer Science and Engineering

Indian Institute of Technology Kharagpur

Spring Semester, 2024-25

April 20, 2025

DECLARATION

I certify that

(a) The work contained in this report has been done by me under the guidance of my supervisor.

(b) The work has not been submitted to any other Institute for any degree or diploma.

(c) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

(d) Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Date: April 20, 2025 (Aman Sharma)

Place: Kharagpur (20CS30063)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

KHARAGPUR - 721302, INDIA

CERTIFICATE

This is to certify that the project report entitled "**XPLOG++: Dynamic Observability for Distributed Edges**" submitted by **Aman Sharma** (Roll No. 20CS30063) to Indian Institute of Technology Kharagpur towards partial fulfilment of requirements for the award of degree of Master of Technology in Computer Science and Engineering is a record of bona fide work carried out by him under my supervision and guidance during Spring Semester, 2024-25.

Professor Sandip Chakraborty
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur
Kharagpur - 721302, India

Date: April 20, 2025

Place: Kharagpur

Abstract

Name of the student: **Aman Sharma** Roll No: **20CS30063**

Degree for which submitted: **Master of Technology**

Department: **Department of Computer Science and Engineering**

Thesis title: **XPLOG++: Dynamic Observability for Distributed Edges**

Thesis supervisor: **Professor Sandip Chakraborty**

Month and year of thesis submission: **April 20, 2025**

This thesis enhances XPLOG, an eBPF-based framework generating causally consistent logs for distributed systems, targeting 6G Compute-on-Demand (CoD) environments. We introduce dynamic logging, significantly reducing overhead by selectively monitoring syscalls, and log multiplexing, enabling efficient multi-service log handling with minimal performance impact. Performance evaluations confirm the efficiency of these features. Furthermore, we propose a CoD architecture utilizing Kubernetes for orchestration, standard monitoring tools (Prometheus/Grafana) for alerting and scaling, and CAMARA-compliant APIs. Within this architecture, the enhanced XPLOG serves as a crucial tool for deep diagnostics and root cause analysis, complementing standard metrics. This work delivers both an optimized observability framework and a practical architecture for building manageable and observable 6G edge applications.

Acknowledgements

I would like to thank my guide, Dr. Sandip Chakraborty for his exceptional guidance and support, without which this project would not have been possible. He has always motivated me to explore as much as I can and encouraged me to look into multiple papers from reputed conferences. During the duration of the project, sir has always helped me focus on areas I enjoy working on. He guided me to perform various experiments and try as many ideas as possible in the time frame. He has always been accepting and supportive of the problems I faced during the project.

Finally, I must express my very profound gratitude to my parents for providing me with unfailing support and continuous encouragement throughout my years of study and through the process of researching and writing this thesis. This accomplishment would not have been possible without them. Thank you.

Aman Sharma

Contents

Declaration i

Certificate ii

Abstract iii

Acknowledgements iv

Contents v

1 Introduction 1
- 1.1 Causal Consistency in Distributed Logging . . . 2
- 1.2 eBPF Tracepoints for Syscall Monitoring . . . 3
- 1.3 Compute-on-Demand for 6G Applications . . . 5
- 1.4 Challenges . . . 6
  - 1.4.1 Ensuring Causal Consistency in Distributed Systems . . . 6
  - 1.4.2 Efficient Resource Management in CoD . . . 7
  - 1.4.3 Integration with Existing Infrastructure . . . 7

2 Related Work 8

3 XPLOG++ System Architecture and Enhancements 11
- 3.1 Application and Service Architecture . . . 11
- 3.2 Logging Framework Overview . . . 12
  - 3.2.1 XPLOG Agent . . . 13
  - 3.2.2 XPLOG Collector . . . 14
- 3.3 Data Collection with eBPF . . . 14
  - 3.3.1 Log Information Enrichment . . . 15
- 3.4 Handling Causal Consistency Across Hosts . . . 16
  - 3.4.1 eBPF Ring Buffer and Atomic Event Logging . . . 17
- 3.5 Dynamic Logging . . . 18
  - 3.5.1 Motivation . . . 18
  - 3.5.2 Design . . . 19

3.5.3 Implementation Details . . . 20
3.6 Log Multiplexing for Multi-Service/Multi-Tenant Environments . . . 21
3.6.1 Motivation . . . 21
3.6.2 Design Principle . . . 22
3.6.3 Implementation Details . . . 23
3.6.3.1 Considered Log Format Alternatives . . . 23
3.6.3.2 Comparison of Alternatives . . . 24
3.6.3.3 Chosen Approach and Implementation . . . 25

4 6G Compute-on-Demand Architecture and Scheduling 28
4.1 Motivation and Goals . . . 28
4.2 Proposed Compute-on-Demand Architecture . . . 30
4.3 Service Orchestration, Scheduling, and Autoscaling . . . 33
4.3.1 Challenges in Edge Scheduling . . . 33
4.3.2 Considered Scheduling Approaches . . . 33
4.3.3 Adopted Approach: Kubernetes-based Multi-Objective Optimization . . . 34
4.3.4 SLA Adherence via Autoscaling . . . 35
4.3.5 Monitoring Integration (Prometheus/Grafana XPLOG++) . 36
4.3.6 Hierarchical Orchestration Model . . . 38
4.3.7 CAMARA API Compliance . . . 38
4.3.8 Integrated System Workflow . . . 39

5 Performance Evaluation 41
5.1 Introduction . . . 41
5.2 Experimental Setup . . . 42
5.3 Methodology . . . 43
5.4 Results . . . 44
5.5 Analysis and Discussion . . . 45
5.6 Conclusion . . . 47

6 Conclusion and Future Work 49
6.1 Conclusion . . . 49
6.2 Future Work . . . 49

Bibliography 51

Chapter 1

Introduction

In this thesis, we present additions to XPLOG, a logging framework that produces a causally consistent ordering of logs collected from multiple systems. We test the compatibility of the framework with the proposed Compute on Demand environment and make it more efficient for practical production usage. We add Dynamic Logging, which allows the user to choose which syscalls they want to be logged for a particular system, allowing lower resource consumption and providing greater flexibility. By leveraging eBPF (Extended Berkeley Packet Filter) technology, our framework captures system-level events with minimal overhead, allowing efficient and real-time logging with rich context even in high-performant scenarios. We also add Log Multiplexing, which allows XPLOG to be used alongside other services for more comprehensive monitoring of edge applications. We also propose a Compute on Demand architecture for 6G applications, which encompasses things like end-user APIs, scheduling algorithms and runtime SLA fulfillment.

In distributed systems, logging is a critical tool for monitoring, debugging, and performance optimization. Logging involves systematically recording system events to

provide insights into system behaviour, detect anomalies, and enhance overall observability. In a traditional single-system architecture, this process is as trivial as collecting events along with their timestamps from different applications; however, distributed architectures pose unique challenges for log collection, aggregation, and analysis. With the rise of dynamic, resource-constrained applications—particularly those in next-generation networks such as 6G—logging requirements have become more complex and demanding. With the research going into implementing a hyper-local Compute-on-Demand architecture for 6G technology, a framework that generates logs that are easy to read and meaningful is of utmost importance and one of the fundamental building blocks of this new age of computing. These applications require not just high throughput and low latency, but also causally consistent logs that maintain the logical order of events across distributed subsystems.

The need for a causally consistent logging framework is essential for accurate debugging and traceability in distributed environments. Causal consistency, a relaxed consistency model, ensures that events are recorded in the order in which they causally affect each other, rather than simply by their timestamp. This approach allows systems to capture the logical dependencies between events, which is vital for analyzing distributed transactions or workflows in 6G applications. Traditional logging frameworks lack this capability, which limits their utility for distributed, real-time applications.

1.1 Causal Consistency in Distributed Logging

Causal consistency is one of several models of consistency used to manage data in distributed systems. In any distributed system, events (such as updates or log entries) are often generated across multiple nodes, making it challenging to determine

a globally consistent order for these events. Causal consistency, unlike strict consistency models (e.g., strong or linearizability), doesn't require all events to be ordered the same across every node. Instead, it only enforces the order of events that have a causal relationship—meaning one event may logically depend on another. In layman's terms, it just means if an operation is visible, so should all the operations that this depends on.

Events that are not causally related can occur in any order, which offers greater flexibility in handling distributed updates and improves system performance. These events are known as Independent Events. The resulting model is "weaker" than strict consistency but retains a level of logical coherence that is essential for many applications, and what we end up using for our logging framework.

To achieve causal consistency between events, just the timestamp is enough if it can be ensured that the timestamp accurately represents the order in which those events occurred. In a distributed system, however, since local clocks can drift apart, using just timestamps is not enough to ensure consistency between different systems. Vector Clocks ([1]) are most commonly used in such scenarios. A vector clock is a vector of counters per node, with each entry representing the number of events known to have occurred at each node. These vector clocks are sent from one node to another when any communication between them happens. When a causally dependent event occurs, vector clocks can help ensure it is ordered after its dependencies across the system.

1.2 eBPF Tracepoints for Syscall Monitoring

Extended Berkeley Packet Filter (eBPF) technology is a Linux-based tool that enables the safe, efficient, and high-performance monitoring of kernel-level events.

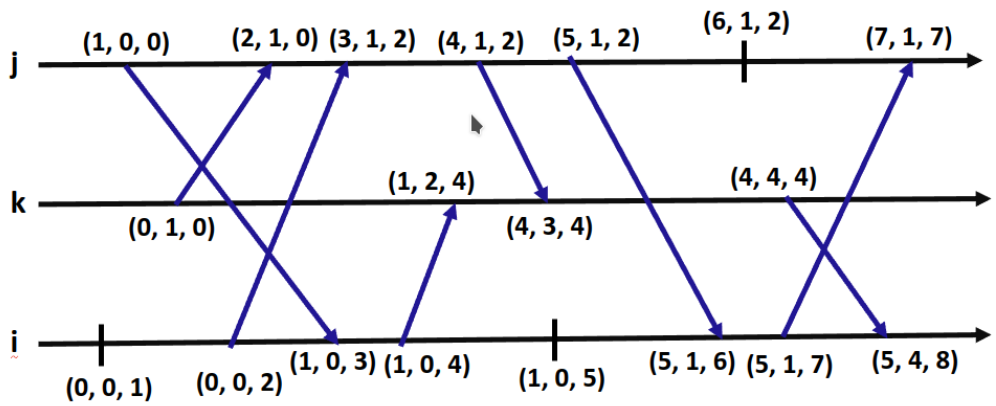

FIGURE 1.1: Example of how vector clock values are propagated between nodes

eBPF allows user-defined programs to be executed within the kernel, facilitating detailed visibility into kernel functions without necessitating extensive modifications, kernel recompilations or overhead. Through tracepoints, eBPF can monitor system calls, enabling the capture of events at the level of system processes and interactions. This capability is relevant for us as it allows for fine-grained data collection directly at the kernel level.

Our framework leverages eBPF tracepoints to monitor system calls, enabling efficient and precise log collection that captures kernel-level activity with minimal performance impact. By focusing on syscall tracepoints, our framework provides detailed insights into the process and thread-level events, which are essential for accurately mapping out system interactions and ensuring causal consistency across multiple nodes. The use of eBPF also minimizes overhead, ensuring that the logging framework remains performant and scalable, particularly suited to the latency-sensitive and resource-constrained environments typical of 6G applications.

1.3 Compute-on-Demand for 6G Applications

Compute on demand (CoD) in 6G is an innovative approach that utilizes the computational capabilities embedded within modern communication towers, effectively turning them into distributed hyperlocal servers. This model is valuable for applications that benefit from low-latency, localized processing, where data processing happens close to the end-user. By activating and allocating these resources only when needed, computing on demand helps optimize infrastructure usage, reduce latency, and deliver high responsiveness for a range of high-performance applications. The key reason why this is important is that these communication towers already exist, with their computing resources going to waste. CoD allows us to utilize these resources and provide a better end-user experience.

For example, a local grocery store could host its inventory and ordering systems on a nearby communication tower, allowing customers to browse and order with near-instant response times. Other applications that could benefit from CoD include smart city infrastructure, AR/VR applications, autonomous vehicles, and healthcare services.

The main advantages of using CoD are:

- **Efficient Resource Utilization:** Since this model transforms communication towers into dual-purpose assets—serving both communication and computation needs—thereby making optimal use of the existing infrastructure. Compute on demand capitalizes on otherwise underused resources, such as idle processors and storage in these towers, improving overall network efficiency without significant hardware investment.

- **Ultra-Low Latency:** By enabling data processing close to the end-user, compute on demand dramatically reduces latency, making it suitable for applications that require real-time responsiveness. This is critical for autonomous driving, AR/VR, and other applications where delays of just milliseconds can impact user experience and functionality.

- **Enhanced Data Privacy and Security:** Processing data locally minimizes the need for transmitting sensitive data across long distances or centralized servers, reducing exposure to potential data breaches. This is especially relevant for applications involving personal or location-based data, such as healthcare and autonomous systems, where maintaining data privacy and security is extremely important.

- **Bandwidth Savings:** By reducing the amount of data sent to distant data centres, CoD helps reduce network load and lowers bandwidth consumption. This localized processing approach reduces congestion on backbone networks of the internet, allowing for more efficient data management across the entire network.

1.4 Challenges

The main challenges in implementing such a logging framework and mainly Compute-on-Demand are listed below.

1.4.1 Ensuring Causal Consistency in Distributed Systems

Achieving causal consistency across distributed systems is challenging due to the need to accurately track dependencies between events across multiple nodes. Vector

clocks, which are used in our implementation introduce a computational overhead and implementation complexity to the whole system.

1.4.2 Efficient Resource Management in CoD

Managing and allocating resources dynamically across distributed communication towers involves complex scheduling and load-balancing. This challenge is heightened by the need to adapt to real-time demand fluctuations, ensuring efficient resource utilization while maintaining consistent performance across nodes. Since the primary objective of these network towers is to provide internet access, applications might need to be hot-swapped to other towers. Finding a tower with free resources and close to the required location is a complex problem.

1.4.3 Integration with Existing Infrastructure

To leverage our current systems effectively, the logging framework must minimize the need for extensive instrumentation or complex modifications, such as implementing a custom kernel. Our solution needs to operate within the existing infrastructure, avoiding significant upgrades or the need for new hardware while still supporting advanced frameworks like causal consistency mechanisms and Compute-on-Demand.

Chapter 2

Related Work

Understanding distributed system behaviour necessitates robust logging and observability, yet traditional methods face significant challenges. Early approaches to system provenance tracking (2; 3) often relied on static kernel instrumentation or proxies (4; 5), which proved inflexible, difficult to maintain across kernel versions (6), or struggled with the dynamic nature and scale of modern microservice architectures. A critical requirement for analyzing these distributed systems is maintaining causal consistency in logs collected across multiple nodes, ensuring that event sequences reflect true dependencies rather than just potentially skewed timestamps (7). While mechanisms like vector clocks (1) provide a theoretical basis for capturing causality, their implementation can introduce notable overhead and complexity in large-scale, dynamic deployments.

The advent of eBPF has revolutionized kernel observability in Linux (8), providing a safe, performant, and programmable way to monitor system events without kernel modifications or fragile LKMs. Its ability to dynamically attach probes to tracepoints or kprobes enables fine-grained monitoring, leading to its wide adoption in advanced networking (9), security (e.g., Cilium (10)), and performance analysis

tools (e.g., Pixie ([11])). XPLOG++ leverages this eBPF capability specifically for capturing syscall events efficiently. While established logging frameworks like the ELK/EFK stack ([12]; [13]) excel at aggregating application logs, and tools like Tracee ([14]) use eBPF for single-host security tracing, XPLOG++ differentiates itself by focusing on generating causally consistent log streams that correlate system-level events across distributed hosts, enhanced further by the dynamic logging and multiplexing features introduced in this work.

This focus on distributed systems observability is particularly relevant in the context of edge computing ([15]) and Multi-access Edge Computing (MEC) ([16]), which are foundational for latency-sensitive 6G applications ([17]; [18]). The Compute-on-Demand (CoD) model, utilizing compute resources within the network infrastructure ([19]), exemplifies this trend but intensifies challenges related to managing geographically dispersed, heterogeneous, and resource-constrained nodes. Orchestrating microservices in such environments requires sophisticated solutions. While Kubernetes ([20]) provides a powerful foundation, its application at the edge often necessitates lightweight distributions (e.g., K3s ([21]), KubeEdge ([22])) and edge-aware scheduling strategies ([23]) beyond its default capabilities. Concurrently, WebAssembly (WASM) is emerging as a suitable runtime for edge deployments due to its portability, security, and efficiency ([24]). Furthermore, standardization efforts like the CAMARA project ([25]) are crucial for providing consistent APIs for developers interacting with edge platforms.

Our work positions the enhanced XPLOG framework within this evolving landscape. It acts as a specialized observability tool providing deep, causally consistent insights that complement standard metrics-based monitoring (like Prometheus/-Grafana) within a Kubernetes-managed CoD architecture. By leveraging eBPF efficiently, incorporating dynamic logging to manage overhead, supporting multi-service environments via multiplexing, and integrating with an architecture aligned

with industry standards (Kubernetes, CAMARA), XPLOG++ aims to provide the necessary diagnostic capabilities for complex, distributed edge systems without compromising performance or manageability.

Chapter 3

XPLOG++ System Architecture and Enhancements

This chapter describes the architecture of the XPLOG++ framework, designed for observability in distributed edge environments, particularly targeting 6G Compute-on-Demand scenarios. It details the core components, the use of eBPF for efficient log collection, and the mechanisms for ensuring causal consistency. Furthermore, it elaborates on two key enhancements developed during this project: Dynamic Logging, which allows selective syscall monitoring to reduce overhead, and Log Multiplexing, which enables efficient handling of logs from multiple distinct services over a single connection.

3.1 Application and Service Architecture

The XPLOG++ framework is designed to operate within modern distributed systems, aligning with the principles of microservice architecture ([26]; [27]). Applications are expected to be decomposed into modular, isolated services. In the context of our

target 6G CoD environment, these services are run within sandboxed WebAssembly (WASM) runtimes. This approach offers several advantages:

- **Isolation and Security**: Each application or service runs in its own sandbox, minimizing interference and enhancing security.
- **Portability**: WASM allows services to be deployed across different hardware and operating systems with no additional overhead.
- **Dynamic Deployment**: The independence of services facilitates dynamic migration and scheduling across different edge nodes (e.g., communication towers) based on load, latency requirements, or resource availability, which is crucial for CoD.

In addition to WASM-based applications, essential backend services, such as databases or caching layers, are typically deployed within containers (e.g., Docker). This containerized approach ensures that both application logic and supporting services operate autonomously, minimizing inter-dependencies and further enabling flexible deployment and scaling strategies suitable for dynamic edge environments. The separation allows independent movement and management of components, aligning perfectly with the CoD model.

3.2 Logging Framework Overview

To provide effective observability and maintain causal consistency across logs originating from these distributed services, the framework builds upon the core XPLOG architecture ([28]). The system captures logs from individual hosts (both application-level and system-level via syscalls), orders them based on the causal sequence of

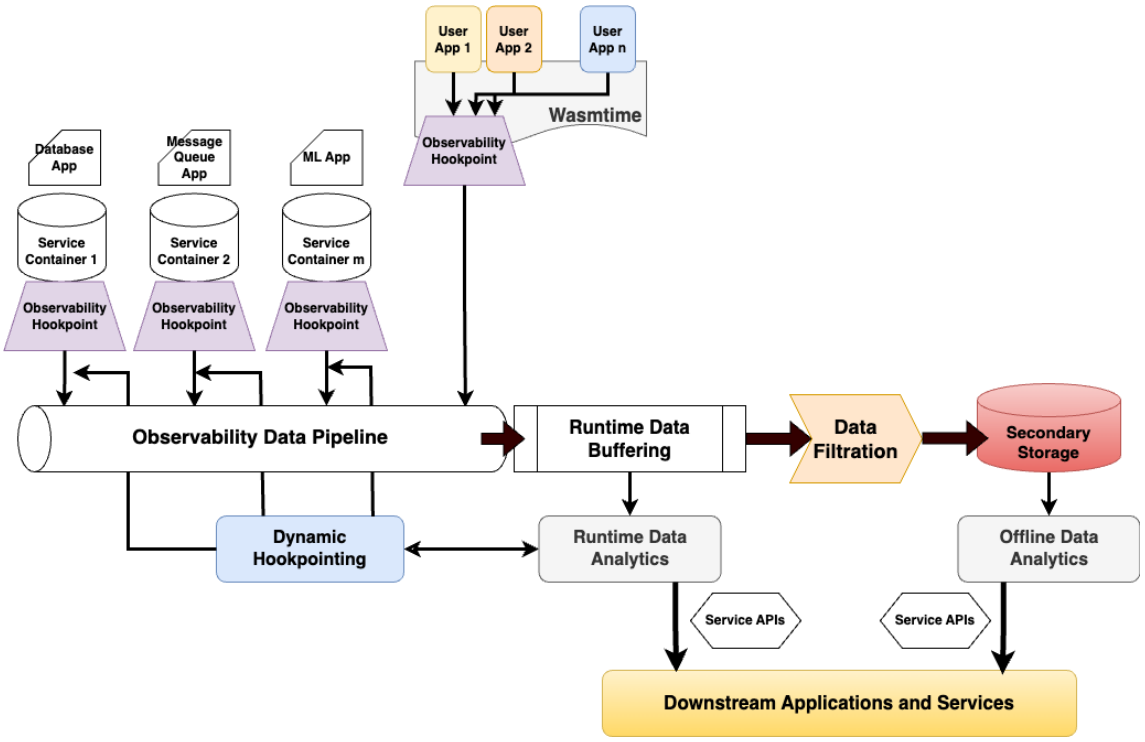

FIGURE 3.1: System Architecture for XPLOG++ Supporting WebAssembly Runtime and Containerized Services

events across microservices, and forwards them in a consistent stream to downstream analysis or storage systems via the XPLOG Collector.

3.2.1 XPLOG Agent

The XPLOG Agent is deployed as a privileged container or process on each monitored host. Its privileged status is necessary to interact with the kernel via eBPF for syscall monitoring. The Agent acts as the primary log collection point on the host and comprises two main parts:

User-Space Component: This component interfaces with the XPLOG Collector and the host OS kernel. System administrators can configure the user-space component to monitor specific syscalls, and it utilizes kernel tracepoints to capture these events. This component also polls logs from the kernel-space component and sends

them to the XPLOG Collector. By using a ring buffer shared with the kernel, it minimizes synchronization overhead and ensures efficient data transfer.

Kernel-Space Component: This component uses eBPF probes to monitor syscall entry and exit points, providing the necessary data to establish causal consistency within each host. The kernel-space component gathers contextual information, including Task Context and Event Context, to uniquely identify each log entry.

3.2.2 XPLOG Collector

The XPLOG Collector is responsible for aggregating log messages from multiple hosts and ensuring causal consistency across the distributed system. It uses vector clocks ([1]) to order logs from different hosts, implementing a logical clock that accounts for inter-host communication events. By tracking cross-host dependencies, the collector can sequence log messages in a causally consistent order, which is critical for applications distributed across multiple hosts. It also segregates logs based on their originating service, which is discussed in the section about Log Multiplexing 3.6.

3.3 Data Collection with eBPF

The framework leverages eBPF (Extended Berkeley Packet Filter) technology to collect system and application logs efficiently and in an application-agnostic manner. eBPF allows for the injection of custom code, called probes, into specific kernel hook points (tracepoints) at runtime. This capability enables real-time data collection and avoids the need for recompiling the kernel or loading kernel modules, making eBPF a lightweight and flexible solution for logging in dynamic environments.

Tracepoints and Probes: The kernel-space component configures eBPF tracepoints at syscall entry and exit events. Entry probes capture syscall arguments relevant to debugging, while exit probes record additional information such as return values, executable file paths, and task contexts. This data is stored in the `pid_args_map` and `pid_exec_map` eBPF maps, which are indexed by thread IDs to facilitate causal consistency.

Causally Consistent Logging with eBPF Maps: The eBPF maps enable the system to link syscalls across different namespaces, maintaining causal ordering for events generated by containers. By using eBPF maps to store and sequence events, the framework efficiently aggregates logs from various sources and preserves causal consistency, allowing seamless correlation of logs across microservices.

3.3.1 Log Information Enrichment

To support comprehensive observability, XPLOG enriches each log entry with additional contextual information across four main categories:

- **Event Context**: Provides syscall-related data such as timestamp, syscall ID, name, and return value. This information is essential for establishing a causal order of events.
- **Task Context**: Identifies the task generating the syscall, including PIDs, TIDs, and namespace IDs, which disambiguates log entries across multiple PID namespaces.
- **Arguments**: Captures syscall arguments relevant to debugging, which vary based on syscall type. This allows for detailed forensic analysis of system events.

- **Artifacts**: Adds information on executable file paths and accessed files, which is useful for tracing application behaviour.

This enrichment is done in the exit probe right before the syscall exits and execution goes back to the user space.

3.4 Handling Causal Consistency Across Hosts

Maintaining causal consistency across logs from multiple hosts is essential for ensuring accurate system observability. XPLOG uses a two-step approach to achieve this:

- **Intra-Host Causal Consistency**: Within a single host, events are ordered based on their execution sequence by the kernel. The eBPF probes capture events close to their execution time. By using an eBPF ring buffer (Section 3.4.1), which acts as a multi-producer, single-consumer queue, the user-space agent reads events in the order they were committed to the buffer by the kernel probes. This naturally preserves the local causal order across different threads and processes on the same host without relying solely on potentially skewed timestamps.

- **Inter-Host Causal Consistency**: When events involve communication between hosts (e.g., microservice A on Host 1 calls microservice B on Host 2), simply merging timestamp-ordered logs is insufficient due to clock skew. XPLOG++ relies on the Collector to establish cross-host causal order. This is achieved by instrumenting communication points (e.g., network send/receive syscalls) and propagating vector clocks along with messages. The Collector uses these vector clocks to reconstruct a partial order reflecting causal dependencies across hosts before outputting the final log stream.

3.4.1 eBPF Ring Buffer and Atomic Event Logging

Efficiently transferring log data from the high-frequency kernel context to user-space is critical. XPLOG++ utilizes the 'BPF_MAP_TYPE_RINGBUF' provided by the Linux kernel.

- **Efficiency**: The ring buffer offers a lock-free, multiple-producer, single-consumer mechanism for high-throughput data transfer between kernel and user space.
- **Ordering**: As mentioned, events are consumed from the ring buffer in the order they were enqueued by the eBPF probes, preserving the intra-host execution order.
- **Reduced Jitter**: It avoids the performance jitter associated with older mechanisms like perf buffers under high load.
- **Atomic Submission**: In eBPF probes we prepare the full log record and then submit it atomically to the ring buffer, ensuring complete log entries are transferred.

The user-space Agent polls this buffer, retrieves batches of log records, and forwards them to the Collector.

3.5 Dynamic Logging

3.5.1 Motivation

In highly distributed systems, logging can introduce substantial overhead, particularly when logging is performed on every system call. This overhead becomes especially problematic for latency-sensitive applications, such as database-driven systems, where excessive logging can degrade performance. By implementing *dynamic logging*, our framework selectively generates logs only for the syscalls specified by the user. This approach reduces unnecessary log generation and associated latency, improving efficiency and resource usage for applications that only require logs for certain critical events. For instance, in a database application, it may be sufficient to log only the `open` and `close` syscalls for file access, while ignoring `read` and `write` syscalls, thus significantly minimizing the logging overhead.

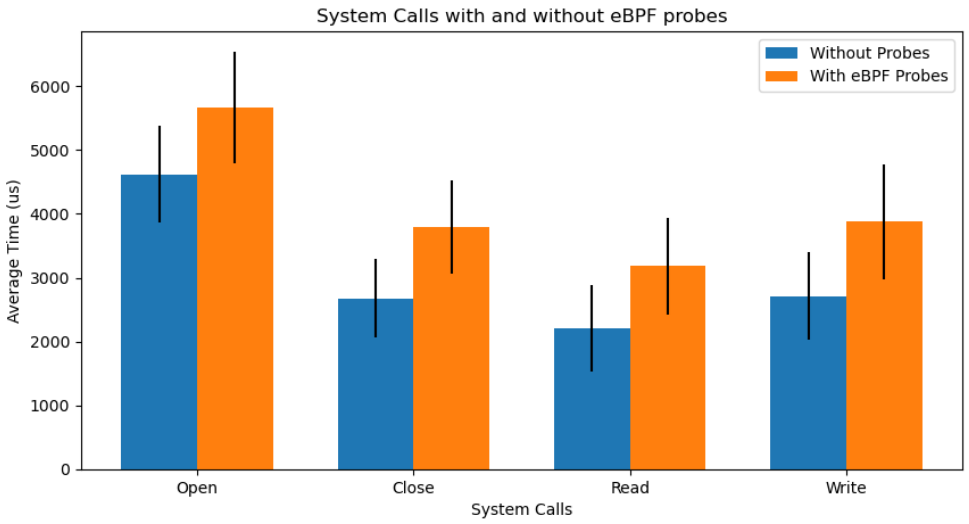

FIGURE 3.2: Comparison of Execution Times for Open, Close, Read and Write Syscalls With and Without eBPF Probes Attached

Performance analysis of eBPF probes shows that attaching probes to syscalls generally increases their execution time by approximately 1.4x as can be seen in 3.2. Given

the substantial performance impact, especially with frequently executed syscalls, selectively logging only specific syscalls is crucial. By dynamically enabling or disabling logging for specific syscalls, we aim to optimize both syscall latency and the network overhead involved in transmitting logs to the XPLOG Collector, effectively balancing system observability with performance.

3.5.2 Design

The dynamic logging mechanism is designed to enable fine-grained control over which syscalls are logged. The key components of this design are:

- **Selective Syscall Logging**: Only syscalls specified by the user are logged. This selective approach is managed via a shared eBPF map (`syscall_required` map), which indicates which syscalls require logging.
- **Syscall Control via User-Space Interface**: A named FIFO pipe acts as a control interface for enabling or disabling syscall logging. Through this pipe, a user-space program writes the syscall number and a boolean flag (indicating whether to log the syscall). This message is read by the eBPF program, which updates the `syscall_required` map accordingly.
- **Efficient Data Handling**: For enabled syscalls, the eBPF probe logs the necessary data by adding it to the eBPF ring buffer, which is then read by the user-space component and sent to the XPLOG Collector. When a syscall is not required, it bypasses logging entirely, avoiding the added latency and network transmission cost.

This design allows for highly flexible, dynamic control over logging, minimizing resource use while retaining essential logging capabilities.

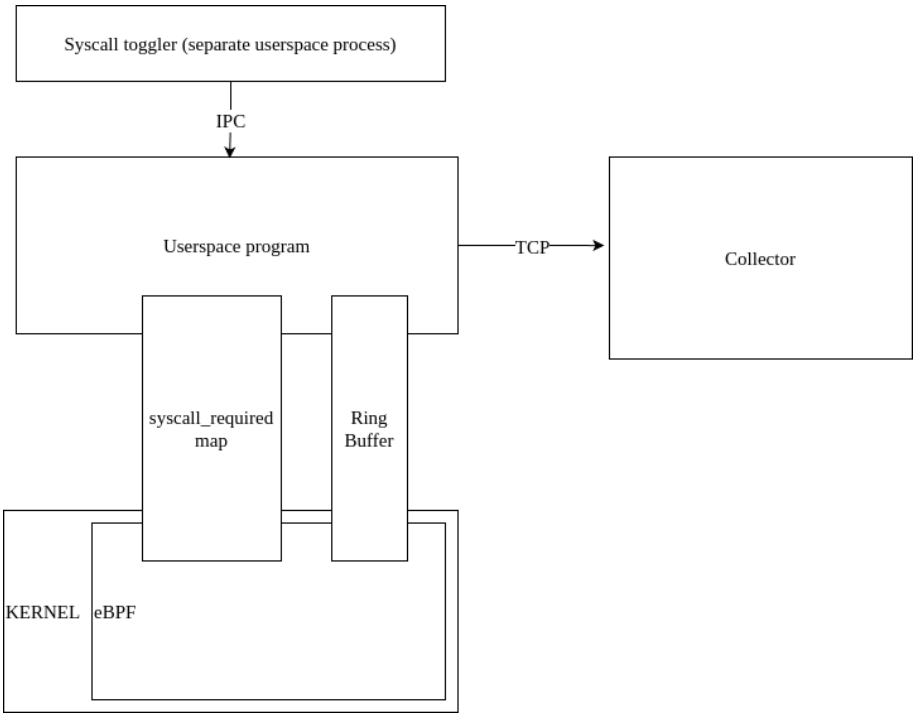

FIGURE 3.3: System Design for Dynamic Logging

3.5.3 Implementation Details

- `syscall_required` **Map**: An eBPF map of type 'BPF_MAP_TYPE_HASH' is defined, mapping syscall numbers (integer key) to a boolean value. This map is accessible by both the user-space XPLOG Agent and the kernel-space eBPF probes.

- **Control Interface (Named FIFO Pipe)**: A standard Linux named pipe (FIFO) is created by the XPLOG Agent user-space process. A separate command-line tool or control script allows the administrator to interact with this pipe.

  – To enable logging for syscall 'N', the tool writes a message like '"N 1"' to the FIFO.

  – To disable, it writes '"N 0"'.

The XPLOG Agent's user-space component continuously reads from this FIFO. Upon receiving a message, it parses the syscall number and the flag, and then updates the corresponding entry in the 'syscall_required' eBPF map using libbpf or equivalent eBPF library functions.

- **eBPF Probe Modification**: Inside the eBPF syscall exit probe function:
  1. The syscall number is obtained from the context.
  2. A lookup is performed in the 'syscall_required' map using the syscall number as the key.
  3. If the map lookup fails (e.g., map not yet populated for this syscall, default to enabled or disabled based on policy) or returns a value indicating "disabled" (e.g., 0), the function returns immediately, skipping log generation.
  4. If the value indicates "enabled" (e.g., 1), the probe proceeds with gathering context, enriching the data, and submitting the log record to the ring buffer as usual.

This implementation provides a flexible and low-overhead mechanism for runtime control over logging granularity.

3.6 Log Multiplexing for Multi-Service/Multi-Tenant Environments

3.6.1 Motivation

In typical microservice deployments, a single host might run multiple independent applications or services, potentially managed by different teams or requiring different

downstream log processing pipelines. Without multiplexing, the XPLOG Agent would either:

- Mix all logs into a single stream, making it difficult for the Collector or downstream systems to attribute logs to the correct originating service.
- Require multiple Agent instances or complex filtering rules.
- Require multiple connections between the Agent and Collector, increasing resource consumption and management complexity.

Log multiplexing addresses this by allowing a single Agent instance to collect logs from various services on the host and transmit them over a single connection to the Collector, while preserving the identity of the originating service.

3.6.2 Design Principle

The core design principle is to prepend a service identifier to each log record before it is transmitted by the Agent. The Collector then uses this identifier to route the log record to the appropriate handler or downstream topic.

- **Service Identification**: Each distinct service or application monitored on the host needs a unique identifier known to both the Agent and the Collector. This could be configured manually, discovered via container orchestration metadata, or based on other criteria.
- **Log Tagging**: The Agent determines the service identifier for each log event (based on the PID/container of the process generating the syscall, for our current implementation this is just the service, i.e., XPLOG logs are ID:0 and other services like logging system metrics, etc can have other IDs) and prepends this ID to the log data before sending it to the Collector.

- **Demultiplexing at Collector**: The Collector reads the service identifier from the beginning of each received log message and forwards the remaining log data to a service-specific processing pipeline or handler.

3.6.3 Implementation Details

Implementing log multiplexing requires embedding a service identifier (‘ServiceID‘) within the data stream flowing from the Agent to the Collector. Several approaches were considered for modifying the log format to accommodate this identifier, each with different trade-offs regarding performance, complexity, and flexibility.

3.6.3.1 Considered Log Format Alternatives

We evaluated three primary methods for tagging logs with their ‘ServiceID‘:

1. **JSON Encapsulation**: Each log message (regardless of its original format) could be wrapped in a JSON object, with the ‘ServiceID‘ included as a dedicated field (e.g., ‘”service_id”: 12‘).
    - *Pros*: Human-readable, leverages standard JSON parsers like the nlohmann JSON library for C++.
    - *Cons*: Introduces significant overhead. The Collector would need to parse the entire JSON structure of every incoming log message just to extract the ‘ServiceID‘ for demultiplexing. Network overhead also increases due to JSON’s verbosity. Furthermore, it forces a JSON structure even if the original service logs are in a different format (which would require wrapping the original log as a string within the JSON, adding more overhead).

2. **Fixed-Length Binary Header (TCP-like)**: Define a rigid binary header structure prepended to each log message, containing fields like ‘ServiceID‘, ‘LogLength‘, checksums, etc., at fixed offsets.

   - *Pros*: Extremely fast parsing at the Collector for demultiplexing (we simply read bytes at known offsets). Minimal parsing overhead for routing.
   - *Cons*: High implementation complexity. Defining, documenting, versioning, and ensuring adherence to a strict binary format is very challenging. It offers less flexibility if the underlying service log formats change frequently and can be wasteful if header fields are not always needed or if logs need padding to fit fixed structures. While it is potentially the optimal solution for extreme performance in a final deployment, it was deemed unnecessarily complex for our current use case.

3. **Prepended ServiceID Byte(s)**: Prepend a small, fixed-size identifier (e.g., 1 or 2 bytes) directly before the original, unmodified log data.

   - *Pros*: Minimal parsing overhead at the Collector (read only the first byte(s) to determine the service). Very low network overhead (only 1-2 extra bytes per message). High flexibility, as the original log payload format is completely untouched and can vary per service. Simple to implement. A single byte allows for 256 distinct services, often sufficient, and easily expandable to two bytes (65,536 services) if needed.
   - *Cons*: The prepended ID is not directly human-readable without context. The number of services is limited by the size of the ID field (though this will be more than sufficient in any real-world deployment).

3.6.3.2 Comparison of Alternatives

The trade-offs are summarized in Table 3.1.

Table 3.1: Comparison of Log Format Alternatives for Multiplexing

Criterion	**JSON**	**Binary Header**	**Prepended Byte**
Parsing Overhead (Demux)	High	Very Low	Low
Network Overhead	High	Low-Medium	Very Low
Implementation Complexity	Low-Medium	High	Low
Payload Format Flexibility	Medium	Low	High
Human Readability (Header)	High	Low	Low
Service Limit	Unlimited	Defined by Field	256 / 65536

3.6.3.3 Chosen Approach and Implementation

Considering the trade-offs, particularly the desire to minimize overhead while maintaining flexibility and implementation simplicity, the **Prepended ServiceID Byte** approach was chosen. It strikes an effective balance, avoiding the significant parsing penalty of JSON and the implementation burden of a fully custom binary format, while still allowing efficient demultiplexing and preserving the original log format integrity for each service.

The specific implementation follows this chosen approach:

- **Log Format Modification**: The data format transmitted from Agent to Collector now follows:

  ```
  Log: [ServiceID (1 byte)] [Original XPLOG Log Data (Variable Length)]
  ```

  A single byte is currently used, allowing up to 256 unique service identifiers. This ‘ServiceID‘ mapping (ID number to service name/metadata) is maintained across the Agent and the Collector, so the handlers for the same service are used at both ends.

- **Agent Logic**:
  - **Service Mapping**: The Agent determines the ‘ServiceID‘ for each log record based on the service it’s coming from. Each service is given a specific ID which is preallocated and appropriate log function is passed to the handler.
  - **Prepending ID**: After retrieving a log record from the eBPF ring buffer or from any other service running either in user or kernel space, the user space component of the Agent uses the log function provided to that service’s handler when execution is passed to it, and this logger automatically prepends the service’s ID to it’s log, which maintains security as the service itself cannot tamper with the serviceIDs.
- **Collector Architecture (Object-Oriented Refactoring)**: The Collector’s receiving component was refactored to efficiently handle demultiplexing based on the prepended byte, using an object-oriented design for extensibility:
  - **Central Collector**: Listens for incoming data from Agents on a single port/socket.
  - **Demultiplexer**: Reads the first byte from the incoming data stream to identify the ‘ServiceID‘.
  - **Handler Registry**: A registry (implemented as a hash map) maps the integer ‘ServiceID‘ (from the first byte) to specific handler objects. Signature: ‘Map<Byte, LogHandler>‘.
  - **Service-Specific Handlers**: Each handler class implements a common **LogHandlerInterface** (with a processLog(char* logData) method). The demultiplexer finds the appropriate handler using the registry and passes the remaining log data (i.e., the original log message, excluding the first byte) to that handler’s **processLog** method.

- **Extensibility**: This design allows new services and their corresponding log handlers to be added easily, just by implementing a new handler class and updating the Collector's configuration to register the new 'ServiceID' and its handler. This greatly improves modularity and maintainability compared to monolithic handling logic.

This multiplexing capability, implemented with minimal overhead, significantly enhances the practicality of XPLOG++ in complex, multi-tenant, or multi-service edge environments by streamlining log transport and enabling service-specific processing downstream.

Chapter 4

6G Compute-on-Demand Architecture and Scheduling

4.1 Motivation and Goals

The evolution towards 6G networks promises unprecedented connectivity speeds and capabilities, enabling a new generation of applications characterized by ultra-low latency, high bandwidth, and massive device density. Many emerging applications, such as real-time augmented/virtual reality (AR/VR), autonomous vehicle coordination, remote robotics, and tactile internet services, demand computation to occur extremely close to the end-user to meet stringent latency requirements (17; 18). Relying solely on centralized cloud data centers introduces significant round-trip delays, rendering them unsuitable for these latency-critical tasks.

To address this challenge, the **Compute-on-Demand (CoD)** paradigm uses the distributed infrastructure found in cellular networks. Modern communication towers (base stations) are equipped with non-trivial computational resources (processing power, memory, storage). CoD proposes using these geographically dispersed

resources as hyperlocal edge servers ([19]). By deploying application services directly onto these towers or nearby edge nodes, computation can be performed much closer to the data source and the end-user, dramatically reducing latency.

The primary goals of adopting a CoD architecture within the 6G context are:

- **Ultra-Low Latency**: Minimize network delay by processing data at the extreme edge of the network, close to users and devices.
- **Efficient Resource Utilization**: Leverage existing, often underutilized compute capabilities within the network infrastructure, reducing the need for dedicated edge data centers and optimizing energy consumption.
- **Bandwidth Conservation**: Reduce the amount of raw data transmitted back to centralized clouds by performing initial processing and filtering at the edge.
- **Enhanced Context-Awareness**: Enable applications that rely on localized information (e.g., local traffic conditions for autonomous vehicles, nearby points of interest for AR).
- **Scalability and Flexibility**: Allow services to be dynamically deployed, scaled, and migrated across edge nodes based on real-time demand and network conditions, managed through robust orchestration.

Realizing these goals requires a sophisticated architecture for managing and orchestrating these distributed edge resources, integrating advanced scheduling, autoscaling, and standardized APIs, while maintaining deep observability for complex diagnostics.

4.2 Proposed Compute-on-Demand Architecture

We propose a CoD architecture designed to integrate seamlessly with 6G network infrastructure, leveraging industry-standard orchestration tools, enhanced by the deep observability of XPLOG++, and exposing standardized APIs. The architecture, shown in Figure 4.1, incorporates hierarchical control and standard monitoring practices.

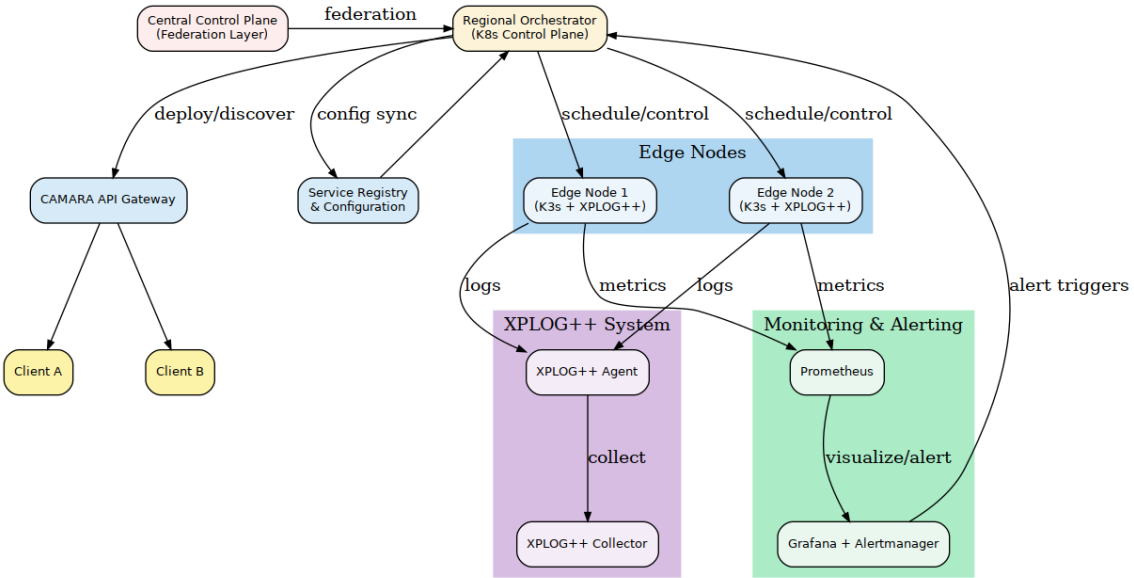

FIGURE 4.1: Proposed 6G Compute-on-Demand Architecture with Kubernetes, Hierarchical Control, Monitoring, and CAMARA APIs

- **Edge Nodes (ENs)**: Distributed compute locations (e.g., 6G towers, micro-DCs) running a lightweight Kubernetes distribution (like K3s, MicroK8s) or managed by a central Kubernetes control plane. They host services in containers or WASM runtimes (managed via shims like containerd-shim-wasmedge). Each EN runs standard monitoring exporters (e.g., node-exporter, kubelet metrics) and an **XPLOG++ Agent** for deep, causal log collection.

- **Hierarchical Orchestration Plane**: Instead of a single central orchestrator, we adopt a hierarchical model:

  - **Regional Orchestrators (Kubernetes Clusters/Control Planes)**: Each manages a cluster of geographically proximate ENs. Responsible for local scheduling, autoscaling, and lifecycle management within its region. ENs connect to their nearest regional orchestrator, reducing control plane latency.
  - **Central Control Plane (Federation Layer)**: Coordinates across regional orchestrators, manages global policies, handles service discovery across regions, and potentially federates clusters.

  This hierarchy improves scalability and resilience.

- **Monitoring and Alerting System**:
  - **Metrics Collection (e.g., Prometheus)**: Aggregates time-series metrics (CPU, memory, network, custom application metrics) from exporters on ENs and services. Deployed regionally, potentially federated centrally.
  - **Visualization & Alerting (e.g., Grafana, Alertmanager)**: Provides dashboards for visualizing metrics and defines alerting rules based on thresholds or patterns (e.g., SLA breaches like high latency, low throughput, high error rate). Alerts can trigger automated actions, including scaling.

- **XPLOG++ System (Agent & Collector)**: Operates alongside the standard monitoring stack.
  - **Agents**: Collect fine-grained, causally consistent syscall/application logs from ENs, using multiplexing (Section 3.6) to tag logs by service (‘ServiceID‘).
  - **Collector**: Aggregates logs, ensures global causal consistency, and stores them (e.g., in Elasticsearch, Loki). Primarily used for **deep debugging,**

root cause analysis of complex distributed issues, security auditing, and feeding very specific, hard-to-get metrics (e.g., fine-grained I/O patterns) back into the main monitoring system if needed, rather than being the primary driver for scheduling/scaling.

- **Service Registry & Configuration**: Stores service definitions (container images, WASM modules), deployment configurations (Kubernetes manifests), resource requirements/limits, HPA/VPA configurations, and SLA definitions.
- **CAMARA-Compliant API Gateway**: Exposes standardized APIs to developers and applications, abstracting the underlying infrastructure complexity. Key relevant APIs include:
  - **Quality on Demand (QoD)**: Allows applications to request specific network QoS characteristics (though implementation depends on underlying network capabilities).
  - **Edge Application Management**: Provides APIs for deploying, managing lifecycle (start, stop, update), and discovering edge applications (consistent with CAMARA's direction).
  - **Application Endpoint Discovery**: Helps clients find the appropriate edge endpoint for a given service, directing them to the closest/best performing instance managed by the orchestrator.
- **Application Clients**: End-user devices or systems interacting with edge services via the API Gateway or directly with discovered endpoints.

4.3 Service Orchestration, Scheduling, and Autoscaling

Managing services effectively in the dynamic CoD environment requires robust orchestration that combines intelligent initial placement with continuous adaptation through autoscaling, driven by comprehensive monitoring.

4.3.1 Challenges in Edge Scheduling

Scheduling services onto resource-constrained, geographically dispersed, and unreliable Edge Nodes presents unique challenges compared to traditional cloud scheduling. Key difficulties include:

- Heterogeneous resources and varying node capabilities.
- Strict latency requirements tied to user location.
- Dynamic fluctuations in workload and network conditions.
- Need for high availability despite potential node failures.
- Optimizing for possibly conflicting goals (latency, resource usage, energy efficiency).

4.3.2 Considered Scheduling Approaches

Several advanced scheduling paradigms were initially considered:

- **Swarm Intelligence (e.g., Particle Swarm Optimization, Ant Colony Optimization)**: These metaheuristics can explore complex solution spaces

effectively. However, they often require significant tuning, can have slow convergence times, and their decision-making process can be opaque, making debugging difficult in a production environment.

- **Reinforcement Learning (RL)**: RL agents could learn optimal placement strategies over time by interacting with the environment. Drawbacks include the need for extensive training data (or long online learning phases), sensitivity to changes in the environment (requiring retraining), and the complexity of defining appropriate state representations, action spaces, and reward functions for the edge scheduling problem. This approach was not chosen due to resource constraints, since we want to run the algorithm itself in every region and fairly often, we want it to be lightweight. Moreover, explainability is extremely important for scheduling decisions because they may result in subpar performance and the need may arise for debugging or changing the heuristics for specific workloads, which is extremely hard to do for a black-box algorithm like RL.

4.3.3 Adopted Approach: Kubernetes-based Multi-Objective Optimization

Given the challenges of the alternative approaches and the desire to leverage mature, well-understood technologies, we have decided to adopt an approach based on the scheduling capabilities inherent in Kubernetes (or a compatible orchestrator). Kubernetes provides a robust framework for multi-objective optimization through its scheduler's filtering and scoring mechanisms:

- **Filtering**: The Kubernetes scheduler automatically filters nodes based on hard constraints defined in the service's deployment manifest, such as:

  - Resource Requests (CPU, memory).
  - Node Selectors (matching node labels).
  - Taints and Tolerations (preventing scheduling on unsuitable nodes unless explicitly tolerated).
  - Volume Affinity/Mountability.

- **Scoring (Prioritization)**: Feasible nodes are then scored based on various priority functions designed to optimize for different goals, such as:
  - Spreading pods across nodes/zones (anti-affinity).
  - Packing pods onto fewer nodes (bin packing, for resource efficiency).
  - Node Affinity (preferring specific nodes).
  - Resource Availability (preferring nodes with more free resources).

- **Extensibility**: The scheduler's priorities and filtering predicates can be customized or extended for edge-specific requirements (e.g., adding network latency as a scoring factor via custom plugins).

By carefully defining resource requests/limits, affinity rules, and custom scheduler plugins, we can use Kubernetes to make placement decisions that implicitly balance multiple objectives relevant to the CoD environment.

4.3.4 SLA Adherence via Autoscaling

Initial placement is only part of the solution. Maintaining SLA adherence under varying load requires dynamic resource adjustments. We leverage Kubernetes' built-in autoscaling mechanisms:

- **Horizontal Pod Autoscaler (HPA)**: Automatically scales the number of service replicas (pods) based on observed metrics like CPU utilization, memory usage, or custom metrics (e.g., requests per second, queue length) obtained from the monitoring system (Prometheus). Rules are defined to scale up when load exceeds thresholds and scale down when load decreases, directly impacting throughput and response time. This is less relevant to our usecase since clients are expected to have a good idea for their requirements in the first place which is ensured by creating extensive SLAs.

- **Vertical Pod Autoscaler (VPA)**: Adjusts the CPU and memory requests/limits assigned to individual pods based on historical usage patterns. This helps ensure pods have adequate resources without being over-provisioned, improving resource efficiency. We leverage VPA in our multitenant system, allowing pods running services from different clients on the same Edge Node to scale appropriately in times of heavy load, allowing a fair usage of the resources between the clients.

These autoscalers provide a reactive mechanism to adapt resource allocation to real-time demand, crucial for meeting performance SLAs in the fluctuating edge environment. However these can be combined with predictive models that utilize the data collected from XPLOG to scale in anticipation of resource demand.

4.3.5 Monitoring Integration (Prometheus/Grafana XPLOG++)

Effective autoscaling and orchestration rely on accurate, timely monitoring data. Our architecture integrates standard tools with the specialized capabilities of XPLOG:

- **Metrics for Autoscaling/Alerting (Prometheus/Grafana)**: Prometheus scrapes metrics from node exporters, kubelet, and application-specific exporters (e.g., exposing request latency via a client library). Grafana visualizes these metrics and Alertmanager fires alerts based on predefined rules (e.g., "CPU utilization > 80% for 5 minutes", "request latency > 20ms"). These alerts directly trigger HPA actions or notify operators.

- **Deep Diagnostics (XPLOG++)**: While Prometheus provides aggregate metrics, XPLOG++ provides fine-grained, causally ordered logs. Its role shifts to:

  - **Debugging Failures**: When a service fails or an SLA is breached despite autoscaling, XPLOG++ logs provide the detailed trace (syscalls, application events across distributed components) needed for root cause analysis.
  - **Analyzing Complex Interactions**: Understanding performance bottlenecks in distributed workflows involving multiple microservices across ENs benefits from the causal consistency provided by XPLOG++.
  - **Security Auditing**: Capturing detailed syscall information aids in security forensics.
  - **Custom Metric Feeder**: For highly specific performance indicators not easily exposed via standard metrics (e.g., detailed file access patterns), as well as input to predictive models for resource usage, XPLOG can process its logs to generate custom metrics fed into Prometheus.

This two-tiered approach uses standard metrics for automated control loops (scaling, basic alerting) and specialized logging for deep dives and complex problem diagnosis.

4.3.6 Hierarchical Orchestration Model

To manage a large-scale deployment across numerous ENs, a flat control plane becomes a bottleneck. We employ a hierarchical structure:

- ENs are grouped into regional clusters, each managed by a Regional Orchestrator (Kubernetes control plane). This localizes most scheduling decisions and control traffic.
- EN Kubernetes agents (kubelets) communicate primarily with their regional control plane, reducing wide-area network dependencies for basic operations.
- Regional Prometheus instances collect metrics locally.
- A central control/federation layer coordinates across regions, managing global service definitions, facilitating inter-region service discovery/communication, and aggregating high-level monitoring data using federation capabilities (e.g., Prometheus Federation).

This structure enhances scalability, fault tolerance (regional failures are isolated), and performance of the control plane.

4.3.7 CAMARA API Compliance

To foster interoperability and simplify integration for developers, the platform exposes APIs aligned with the Linux Foundation's CAMARA project standards:

- An API Gateway acts as the entry point.
- It implements APIs relevant to edge deployment, such as:

- **Device Location / Verify Location:** Useful for location-aware scheduling or service discovery.
- **Quality on Demand (QoD):** Translates application requests for network quality into underlying network configurations (if supported by the 6G network fabric).
- **Edge Discovery, Configuration Management:** Aligned with CAMARA's direction for discovering edge zones, deploying applications (translating requests into Kubernetes manifests), managing their lifecycle, and exposing endpoints.

Providing these standard APIs lowers the barrier to entry for developers wishing to leverage the 6G CoD platform.

4.3.8 Integrated System Workflow

Putting it all together, a typical workflow in this revised architecture looks like this:

1. A developer uses a portal that interacts with the CAMARA-compliant API Gateway to request deployment of an edge application (e.g., via an Edge Application Management API call), specifying their requirements and desired region(s).
2. The API Gateway authenticates the request and forwards it to the appropriate Regional Orchestrator (or central plane if spanning regions).
3. The Regional Orchestrator (Kubernetes) uses its scheduler to find suitable EN(s) within its cluster based on the service manifest (resource needs, affinity rules, etc.) and current node status (obtained via kubelet metrics).

4. Kubernetes instructs the chosen EN(s) to deploy the service (pulling images, starting containers/WASM instances).

5. Standard exporters on the EN and within the service pods begin emitting metrics to the regional Prometheus instance. The XPLOG Agent also detects the new service and starts collecting detailed logs, sending them to the XPLOG Collector.

6. As the service runs, Prometheus collects metrics. If load increases, metrics may cross HPA thresholds.

7. HPA detects the threshold breach via Prometheus metrics and instructs the Kubernetes API server to scale up the service deployment (add more replicas). The scheduler places these new replicas.

8. Alertmanager may trigger alerts in Grafana or notify operators if SLAs (defined via alerting rules) are breached despite scaling, or if failures occur.

9. Operators can use Grafana dashboards to view performance trends and XPLOG logs (e.g., via Kibana or a Grafana Loki integration) to perform deep dives for debugging complex issues identified through alerts or poor performance.

10. Clients discover and connect to the appropriate service endpoint, by using CAMARA Application Endpoint Discovery APIs interacting with Kubernetes service discovery mechanisms.

This integrated system leverages standard, robust tooling for orchestration and basic monitoring/scaling, while retaining the deep, causal observability of XPLOG for diagnostics and specialized insights.

Chapter 5

Performance Evaluation

5.1 Introduction

This chapter presents a performance evaluation of the XPLOG framework, focusing specifically on quantifying the impact of the enhancements described in Chapter 3: Dynamic Logging and Log Multiplexing. The goal is to understand the resource utilization and latency characteristics under different configurations to assess their suitability for deployment in resource-constrained edge environments, such as the 6G Compute-on-Demand architecture outlined in Chapter 4. We compare the performance of the baseline XPLOG++ system (logging all syscalls) against configurations utilizing dynamic logging (logging only selected syscalls) both with and without the log multiplexing feature enabled. Key performance metrics measured include CPU usage, memory consumption, generated log file size, and end-to-end log latency.

5.2 Experimental Setup

The performance tests were conducted in a controlled environment designed to simulate realistic edge node conditions and workloads.

- **Hardware**: All tests were run on a machine equipped with AMD Ryzen 5 4600H (6 cores, 12 threads @ 3.0GHz) and 16 GB of DDR4 RAM. For network-related measurements (CPU/Memory under load across network, log file size), the XPLOG Agent and Collector ran on separate machines connected via a 1 Gbps Ethernet LAN. For latency measurements, the Agent and Collector were run on the *same machine* to eliminate network delay and clock synchronization issues, allowing for a precise measurement of the processing overhead within the XPLOG pipeline itself.
- **Software**:
  - Operating System: Ubuntu 22.04 LTS
  - Kernel: Linux 5.15.0
  - XPLOG: Custom build incorporating the dynamic logging and multiplexing features.
  - Container Runtime: Docker Engine v20.10.17.
  - Monitoring Tools: ‘docker stats‘ was used to stream data into a file before processing it.
- **Workload**: To generate significant system call activity relevant to data-intensive applications, a custom C++ program was developed. This program performed a high volume of file I/O operations:
  - Repeatedly opened a set of files.

  - Performed a large number of ‘read()‘ and ‘write()‘ system calls (approximately 1 million read/write pairs per test run) on these files.
  - Closed the files upon completion.

  This workload generates frequent ‘open‘, ‘close‘, ‘read‘, and ‘write‘ syscalls, allowing us to stress the logging framework. The total test duration for metric collection was standardized to 5 minutes per run.

- **XPLOG Configurations Tested**:

  1. **Baseline XPLOG**: Standard XPLOG logging all encountered syscalls (‘open‘, ‘close‘, ‘read‘, ‘write‘, and others generated by the system/workload). No multiplexing.
  2. **XPLOG w/ Dynamic Logging**: Dynamic logging enabled via the FIFO pipe. Only the ‘open‘ and ‘close‘ syscalls were configured to be logged. All the other syscalls were explicitly disabled via the control interface. No multiplexing.
  3. **XPLOG w/ Dynamic Logging + Multiplexing**: Same dynamic logging configuration as above (only ‘open‘, ‘close‘ logged). Log multiplexing feature enabled. Logs generated by the XPLOG++ Agent for the workload were tagged with ‘ServiceID = 0‘ (byte prepended) before being sent to the Collector, which was configured to demultiplex based on this ID.

5.3 Methodology

The following methods were used to measure the key performance metrics for each configuration:

- **CPU & Memory Usage**: These were measured using docker stats and processed to get the maximum and the average usage through the entire duration of the workload. Measurements were taken while the Agent and Collector were running on separate machines connected over the network

- **Log File Size**: The total size in bytes of the output log file generated by the XPLOG Collector after the 5-minute workload completed was recorded. This reflects the volume of data persisted and, indirectly, the network bandwidth consumed. Measurements were taken while the Agent and Collector were running on separate machines.

- **Log Latency**: Measured as the time difference between the timestamp recorded by the eBPF kernel probe upon syscall entry/exit (using 'bpf_ktime_get_ns()') and the timestamp recorded by the XPLOG Collector immediately before writing the corresponding log entry to its output file. To isolate the framework's processing latency and avoid network inconsistencies, these measurements were performed with both the Agent and Collector running on the *same machine*, using the system's monotonic clock. The average latency across a sample of logged events during the test run was calculated.

Each test configuration was run multiple times, and the average values are reported to ensure consistency and mitigate transient fluctuations.

5.4 Results

The performance metrics collected for the three XPLOG++ configurations under the described workload are summarized in Table 5.1.

Table 5.1: Performance Comparison: Baseline vs. Dynamic Logging vs. Dynamic + Multiplexing

Metric	Config I	Config II	Config III
CPU Usage (% Avg)			
XPLOG++ Agent	2.95%	0.25%	0.26%
XPLOG++ Collector	21.10%	5.15%	5.17%
Memory Usage (RSS)			
XPLOG++ Agent	792 KiB	464 KiB	470 KiB
XPLOG++ Collector	350 KiB	350 KiB	350 KiB
Log File Size (Bytes)	5,645,319	156,356	156,356
Log Latency (ms, Avg)	38 ms	10 ms	11 ms

Config I: Baseline XPLOG

Config II: XPLOG /w Dynamic Logging

Config III: XPLOG /w Dynamic Logging & Log Multiplexing

5.5 Analysis and Discussion

The performance measurements summarized in Table 5.1 provide clear insights into the effects of dynamic logging and log multiplexing within the XPLOG framework.

- **Impact of Dynamic Logging (Config I vs. Config II)**: Enabling dynamic logging and restricting monitored syscalls to only open and close results in substantial performance improvements compared to the baseline of logging all syscalls.
  - The Agent CPU usage drops dramatically from 2.95% to 0.25%, demonstrating the efficiency gained by bypassing log generation for the frequent read and write calls within the eBPF probes.

- The Collector CPU usage shows an even more significant reduction, from 21.10% down to 5.15%. This highlights that processing, ordering, and writing a high volume of logs imposes a considerable burden on the collector, which is greatly alleviated by dynamic filtering at the source.
- Agent memory usage (RSS) decreases noticeably from 792 KiB to 464 KiB, reflecting reduced buffering needs. Interestingly, the Collector memory usage remained constant at 350 KiB. The reason for this is that the Collector writes the logs immediately to the output file and does not keep anything in memory, thus we don't see the memory footprint increasingly.
- Log file size sees a massive reduction of over 97% (from 5.6 MB to 156 KB), directly reflecting the vastly lower number of log entries being persisted. This implies significant savings in storage and network bandwidth.
- Average log latency is reduced by about 74% (from 38 ms to 10 ms), indicating much faster end-to-end processing when the system is not saturated with high-frequency logs.

Overall, dynamic logging proves highly effective in mitigating the overhead associated with comprehensive syscall monitoring.

- **Overhead of Log Multiplexing (Config II vs. Config III)**: Comparing the dynamic logging configuration with and without multiplexing isolates the cost of the latter feature.
  - CPU usage increases minimally for both the Agent (0.25% to 0.26%) and the Collector (5.15% to 5.17%), confirming that the logic for prepending the 'ServiceID' byte and demultiplexing at the collector adds negligible computational overhead.

  - Agent memory usage shows a very small increase (464 KiB to 470 KiB), while Collector memory remains unchanged at 350 KiB, further reinforcing that the multiplexing feature itself is lightweight in terms of memory footprint.
  - The measured Log File Size shows no increase between Config II and Config III (both 156,356 Bytes). The reason for this is that the extra byte we prepend to the log is taken away during processing by the Collector, so the final logs that are printed are the same size, which is the expected behaviour.
  - Average log latency shows a small increase from 10 ms to 11 ms. This 1 ms difference is consistent with the minimal extra processing required for handling the 'ServiceID'.

  The results indicate that log multiplexing adds very little performance overhead, making it a practical feature addition.

- **Overall Assessment**: Dynamic logging offers substantial performance benefits, making comprehensive logging feasible in performance-sensitive scenarios by allowing focus on critical events. The log multiplexing feature provides valuable functionality for multi-service environments at a minimal performance cost, especially when combined with dynamic logging. The combination (Config III) represents an efficient and flexible configuration for edge deployments.

5.6 Conclusion

This performance evaluation quantitatively demonstrates the effectiveness of the enhancements made to the XPLOG framework. Dynamic logging significantly reduces CPU usage, memory footprint (primarily agent-side), log volume, and latency

compared to baseline full syscall logging, proving crucial for resource-constrained environments. The introduction of log multiplexing, enabling the handling of multiple services over a single connection, was shown to add only negligible overhead in terms of CPU, memory, and latency. These findings confirm that the enhanced XPLOG framework, particularly when employing dynamic logging with or without multiplexing, provides a powerful yet efficient observability solution capable of delivering deep, causally consistent insights in demanding 6G and edge computing scenarios without imposing prohibitive performance penalties.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

In this thesis, I presented XPLOG++, my eBPF-based observability framework designed to deliver causally consistent logs for 6G Compute-on-Demand. We introduce key enhancements: dynamic logging, which yielded significant performance gains, and log multiplexing for efficient multi-service handling with minimal overhead. We also propose a Kubernetes-based CoD architecture integrating standard monitoring (Prometheus/Grafana) and CAMARA APIs, positioning XPLOG++ as a tool for deep diagnostics. The performance evaluations confirmed the efficiency of these enhancements, establishing XPLOG++ as a practical solution for detailed observability in resource-constrained edge environments.

6.2 Future Work

For future work, there are several directions: integrating resource metrics directly into XPLOG++ logs; developing predictive autoscaling models using the collected

log data and creating advanced Kubernetes scheduling plugins tailored for the edge, out of which some problems are already being worked on.

Bibliography

[1] A. Arora, S. Kulkarni, and M. Demirbas, "Resettable vector clocks," in *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '00, (New York, NY, USA), p. 269–278, Association for Computing Machinery, 2000.

[2] W. U. Hassan, M. A. Noureddine, P. Datta, and A. Bates, "Omegalog: High-fidelity attack investigation via transparent multi-layer log analysis," *Network and Distributed System Security Symposium*.

[3] X. Ge, B. Niu, and W. Cui, "Reverse debugging of kernel failures in deployed systems," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pp. 281–292, 2020.

[4] J. Zeng, Z. L. Chua, Y. Chen, K. Ji, Z. Liang, and J. Mao, "Watson: Abstracting behaviors from audit logs via aggregation of contextual semantics.," in *NDSS*, 2021.

[5] U. Satapathy, R. Thakur, S. Chattopadhyay, and S. Chakraborty, "Disprotrack: Distributed provenance tracking over serverless applications," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*, pp. 1–10, IEEE, 2023.

[6] L. Zhou, F. Zhang, K. Leach, X. Ding, Z. Ning, G. Wang, and J. Xiao, "Hardware-assisted live kernel function updating on intel platforms," *IEEE Transactions on Dependable and Secure Computing*, 2023.

[7] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, p. 558–565, July 1978.

[8] C. Cassagnes, L. Trestioreanu, C. Joly, and R. State, "The rise of ebpf for non-intrusive performance monitoring," pp. 1–7, 04 2020.

[9] M. Bonola, G. Belocchi, A. Tulumello, M. S. Brunella, G. Siracusano, G. Bianchi, and R. Bifulco, "Faster software packet processing on {FPGA}{NICs} with {eBPF} program warping," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pp. 987–1004, 2022.

[10] Isovalent, "Cilium: eBPF-based networking, security, and observability." https://cilium.io, 2020. Accessed: 2025-04-21.

[11] "Pixie: Instant open-source debugging for kubernetes applications." https://px.dev/, 2021. Accessed: 2025-04-21.

[12] Elastic, "Logstash: Collect, Parse, Transform Logs — Elastic." https://www.elastic.co/logstash/, June 2024. [accessed 5-Nov-2024].

[13] Fluent, "Fluentd — Open Source Data Collector — Unified Logging Layer." https://www.fluentd.org/, June 2024. [accessed 5-Nov-2024].

[14] A. Security, "Tracee— GitHub." https://github.com/aquasecurity/tracee, June 2024. [accessed 5-Nov-2024].

[15] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, pp. 1–1, 10 2016.

[16] B. Liang, M. A. Gregory, and S. Li, "Multi-access edge computing fundamentals, services, enablers and challenges: A complete survey," *Journal of Network and Computer Applications*, vol. 199, p. 103308, 2022.

[17] W. Saad, M. Bennis, and M. Chen, "A vision of 6G wireless systems: Applications, trends, technologies, and open research problems," *IEEE Network*, vol. 34, no. 3, pp. 134–142, 2019.

[18] S. A. Abdel Hakeem, H. H. Hussein, and H. Kim, "Vision and research directions of 6g technologies and applications," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 6, Part A, pp. 2419–2442, 2022.

[19] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on multi-access edge computing for internet of things realization," *IEEE Communications Surveys amp; Tutorials*, vol. 20, no. 4, p. 2961–2991, 2018.

[20] C. N. C. Foundation, "Kubernetes documentation." `https://kubernetes.io`, 2024. Accessed: 2025-04-21.

[21] C. N. C. Foundation, "K3s: Lightweight kubernetes." `https://k3s.io`, 2024. Accessed: 2025-04-21.

[22] C. N. C. Foundation, "Kubeedge: Kubernetes native edge computing framework." `https://kubeedge.io`, 2024. Accessed: 2025-04-21.

[23] P. S. Catali, A. Manacero, R. S. Lobato, R. Spolon, and M. A. Cavenaghi, "Comparison of edge computing scheduling algorithms," in *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, pp. 1–6, 2023.

[24] M. Foundation, "WebAssembly system interface (WASI)." `https://wasi.dev`, 2020. Accessed: 2025-04-21.

[25] T. L. Foundation, "Camara: The linux foundation telco api project." https://www.computerweekly.com/blog/Open-Source-Insider/Camara-The-Linux-Foundation-telco-API-project, 2025. Accessed: 2025-04-21.

[26] S. Boucher, A. Kalia, D. G. Andersen, and M. Kaminsky, "Putting the" micro" back in microservice," in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 645–650, 2018.

[27] S. Wang, Z. Ding, and C. Jiang, "Elastic scheduling for microservice applications in clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 1, pp. 98–115, 2020.

[28] S. Chakraborty, "Xplog: A dynamic observability framework for distributed edge applications." 2024.